

Deep Learning

Universal approximation

Boolean functions

- *Any Boolean function of n inputs can be exactly represented with one hidden layer!*

Universal Approximation

- *We can represent any continuous function ($f : \mathbb{R}^m \rightarrow \mathbb{R}^n$) to any desired approximation ($|g(x) - f(x)| < \epsilon$) with a linear combination of sigmoid neurons*
- *In other words, neural networks with a single hidden layer can be used to approximate any continuous function to any desired precision*

Visual Proof

- <http://neuralnetworksanddeeplearning.com/chap4.html>

Universal Approximation

- **universal approximation theorems (UATs)** state that neural networks with a certain structure can, in principle, approximate any continuous function to any desired degree of accuracy.
- Feedforward neural networks with non-polynomial activation function (which is true for common choices like the sigmoid function or ReLU), then the network can act as a "universal approximator."
- A ReLU network with **width at least $d+1$** (where d is input dimension) is a universal approximator when depth is allowed to grow.
- Universality can be achieved either by making a network wide enough with one hidden layer, or by keeping width above a small threshold and making the network sufficiently deep—provided the activation is non-polynomial.

Other models of artificial neuron

- Tanh Neuron

$$\tanh(w \cdot x + b),$$

$$\tanh(z) \equiv \frac{e^z - e^{-z}}{e^z + e^{-z}}.$$

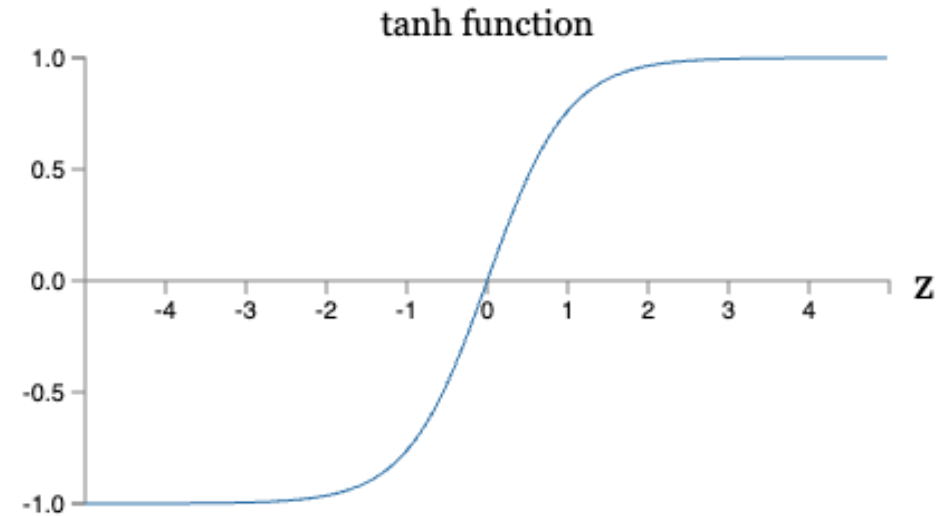
Other models of artificial neuron

- Tanh Neuron

$$\tanh(w \cdot x + b),$$

$$\tanh(z) \equiv \frac{e^z - e^{-z}}{e^z + e^{-z}}.$$

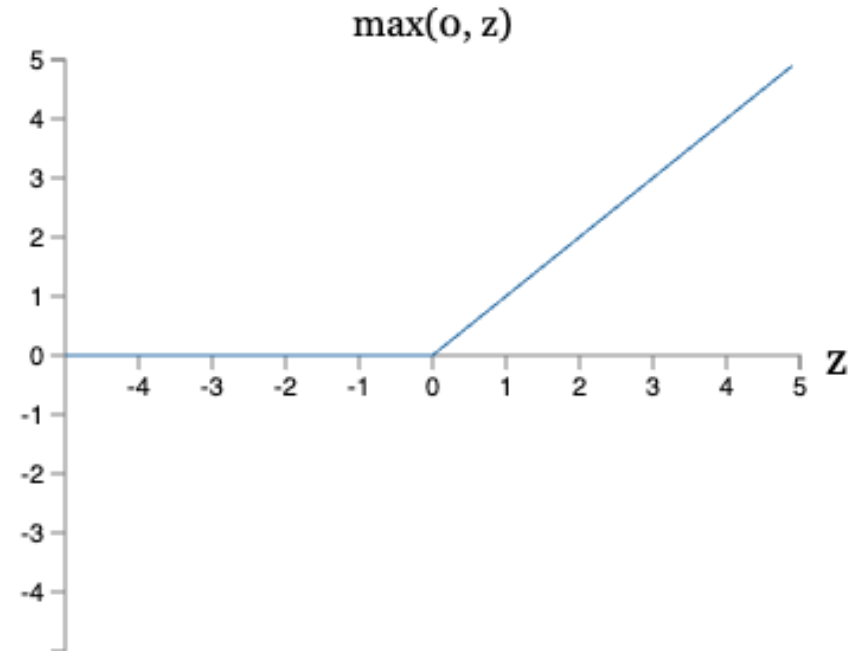
$$\sigma(z) = \frac{1 + \tanh(z/2)}{2},$$



Other models of artificial neuron

- *rectified linear neuron or rectified linear unit (ReLU)*

$$\max(0, w \cdot x + b).$$



Which is the best for any given problem?

Which will result in a network which learns fastest?

Which will give the highest test accuracies?

Sigmoid and Tanh saturates for large x , slows or stalls gradient, vanishing gradient problem

ReLU used in Deep networks, Tanh used in RNNs, LSTMs, Sigmoid in output neurons

Training Multilayer Perceptrons Before Backpropagation

- Before 1986, training multilayer perceptrons (MLPs) was a major challenge
- Lack of efficient credit assignment for hidden layers
- Several alternative learning paradigms were explored

Layer-Wise and Greedy Training

- Train one layer at a time and freeze it
- Artificial **local targets** are defined
- Avoids backpropagating error
- Produces suboptimal internal representations
- Early layers are **not optimized for final task**

Random Weights and Fixed Features

- Hidden layers initialized randomly and kept fixed
- Only output layer is trained
- Requires very large networks for good performance

$$y = W_2 \phi(W_1 x), \quad W_1 \text{ fixed}$$

Reinforcement and Reward-Based Learning

- Learning driven by scalar reward signal
 - Only a **scalar reward** is available, not per-neuron error.

$$\Delta w \propto r \cdot \text{activity}$$

- No gradient or explicit target
- Very slow and high variance learning

Evolutionary and Genetic Algorithms

- Weights optimized via mutation and selection
- No need for differentiability
- Computationally infeasible for large MLPs